

universidade de aveiro



deti

departamento de eletrónica,
telecomunicações e informática

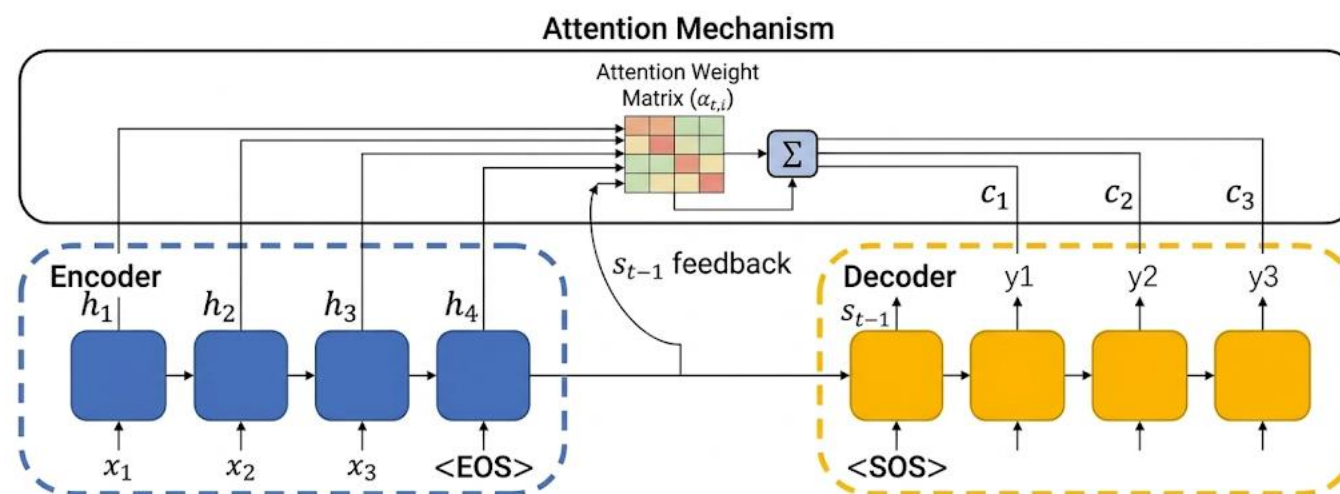
Complements of Machine Learning

2025/2026



What did we saw in the previous class?

- Sequence-to-Sequence models with RNNs.
 - **Bottleneck problem:** The encoder must compress the entire input sentence into a single context vector.
- Attention mechanism:
 - Letting the decoder dynamically decide which input words are most relevant.



Today's Agenda

Topic
1. Language modeling
2. Transformers
3. Adapters

1. Language Modeling

Why do we care?

➤ Traditional way

- Train a neural network from scratch on a specific task (e.g., Sentiment Analysis).
- It learns a very narrow version of English just to solve that one task.

➤ Modern way (Transfer learning)

- A network performs much better on *any* NLP task if it already understands human language.
- We can explicitly train a model on Language Modeling first (using massive text datasets) so it learns how language works before tackling specific tasks.

Modeling the language is a cornerstone subobjective
in any NLP task.

Language modeling

- At its simplest, it is a system trained to predict the next word in a sequence.
 - Given: "The students opened their..." -> Predicts: "books" (80%), "laptops" (15%).



Formally defined by

$$P(w_0, w_1, \dots, w_m) = \prod_{i=0}^m P(w_i | w_0, w_1, \dots, w_{i-1})$$

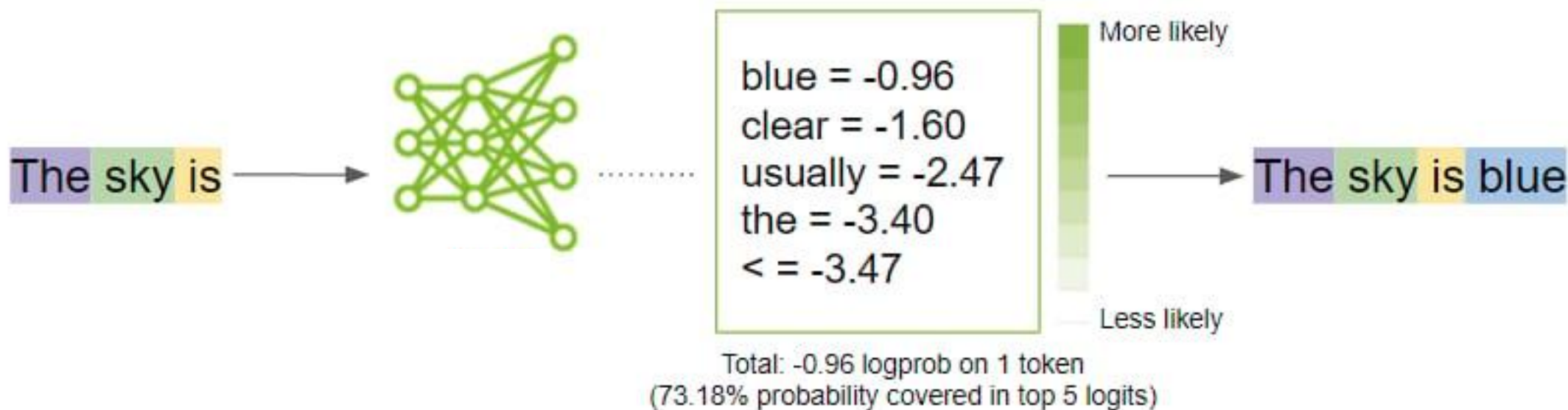


Or

$$\log(P(w_0, w_1, \dots, w_m)) = \sum_{i=0}^m \log(P(w_i | w_0, w_1, \dots, w_{i-1}))$$

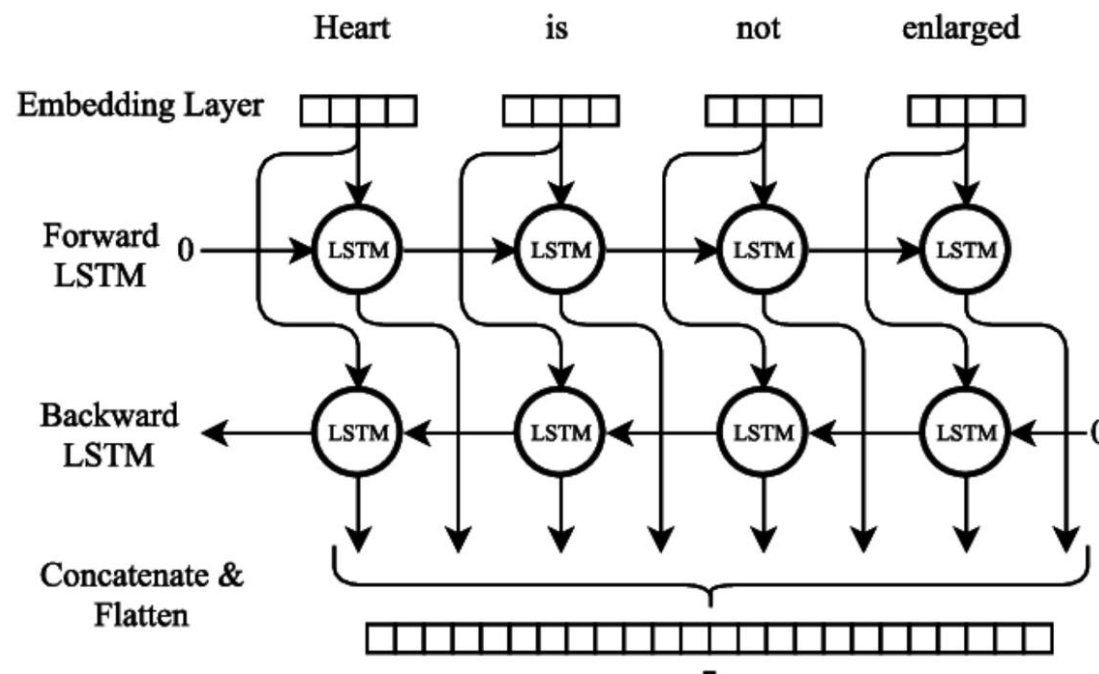
Language modeling

- How we train for this?
 - You can view language modeling as a classification task, where each label to be predicted corresponds to a unique work in the vocabulary.



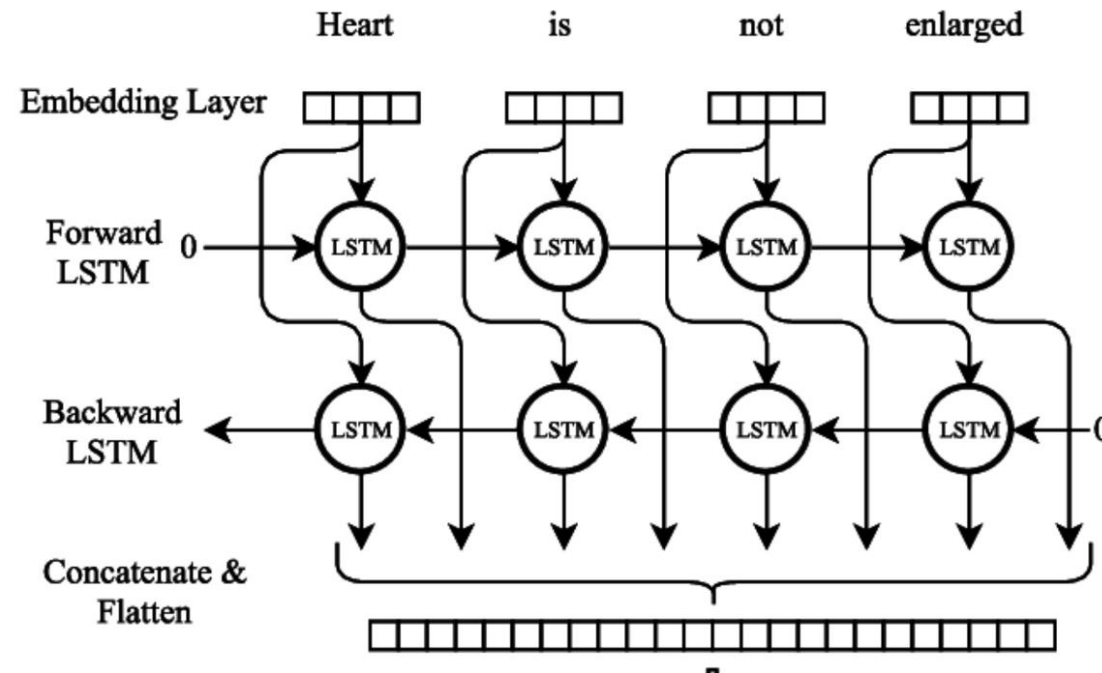
Language modelling: Implementation

- To build these language models, the standard approach was using Recurrent Neural Networks (RNNs) namely LSTMs.
- They process text **sequentially** (from left to right).
- Predict the next word on Trillion of texts!



Language modelling: Implementation

- To build these language models, the standard approach was using Recurrent Neural Networks (RNNs) namely LSTMs.
- They process text **sequentially** (from left to right).
- Predict the next word on Trillion of texts!



Limitation?

Language modelling: Implementation

- Does not scale for trillions of texts!
 - RNN is a neural network that cannot be parallelized on GPUs
 - You cannot compute the word 7 until you have finished computing word 6.

2. Transformer Architecture

Transformer Architecture

➤ Paradigm shift!

What if we could just use feedforward nets?

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

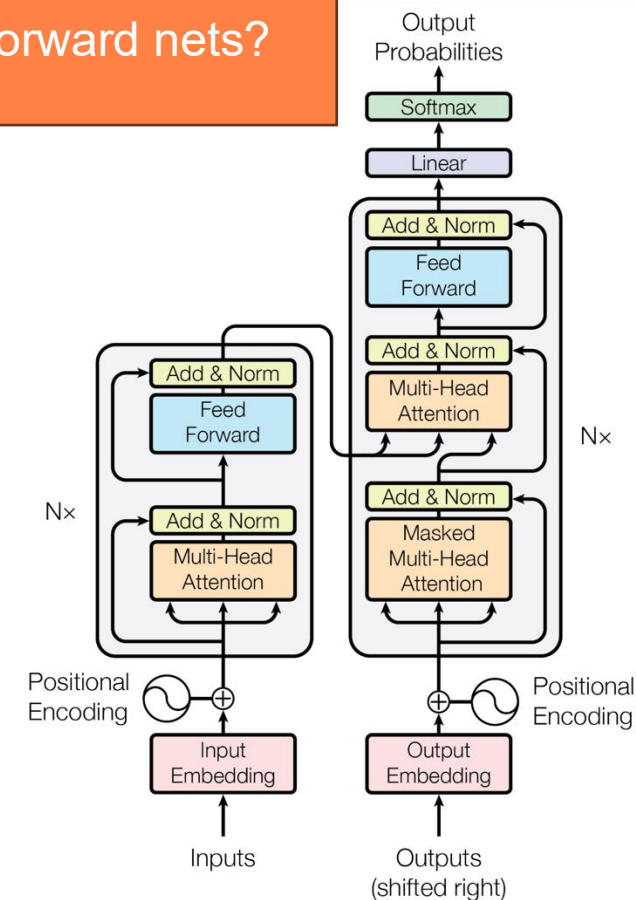
Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaier@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com



Transformer Architecture

➤ Paradigm shift!

What if we could just use feedforward nets?

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

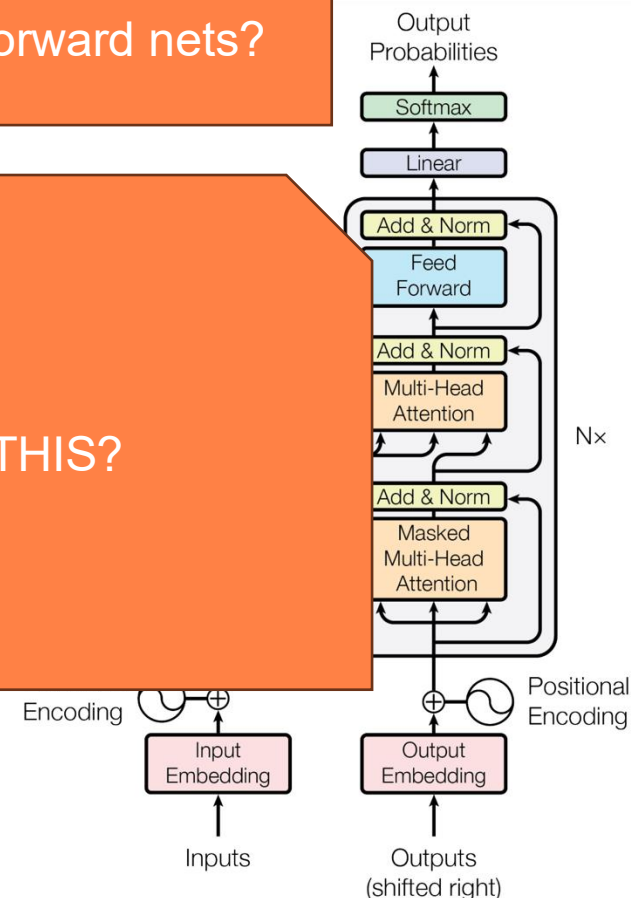
Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

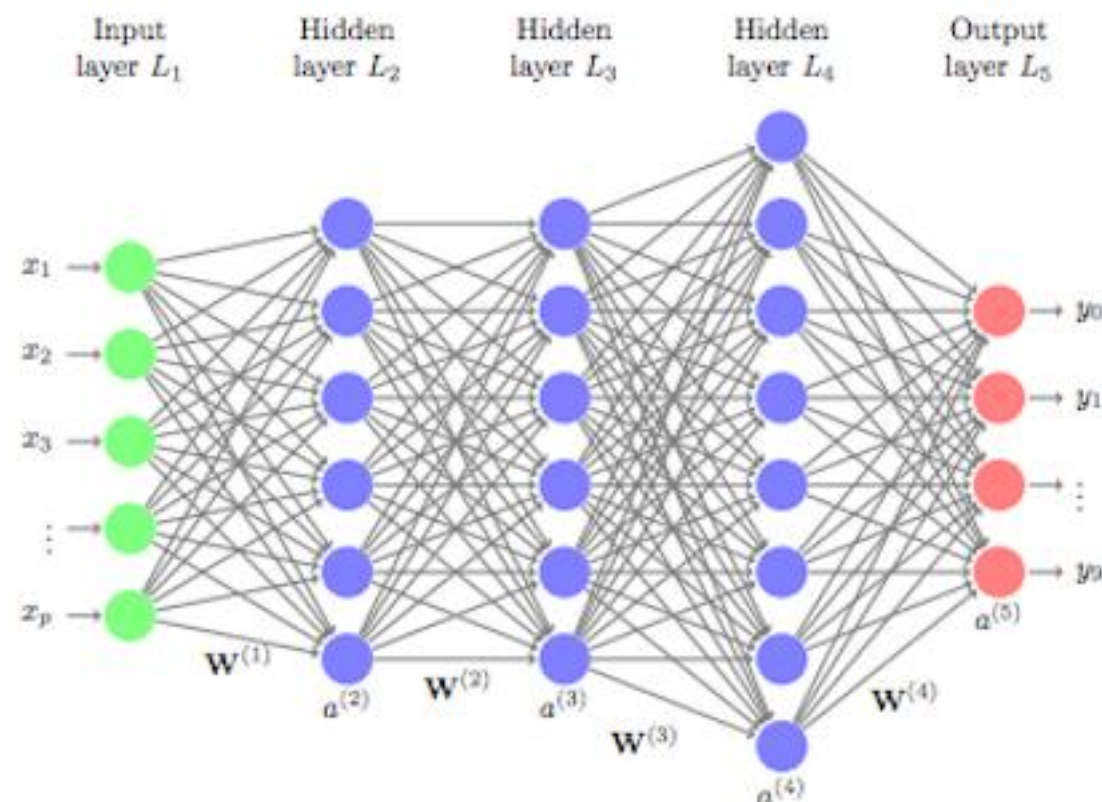
HOW THEY ACHIEVE THIS?



How to process text in NN



➤ Feed-forward neural network?

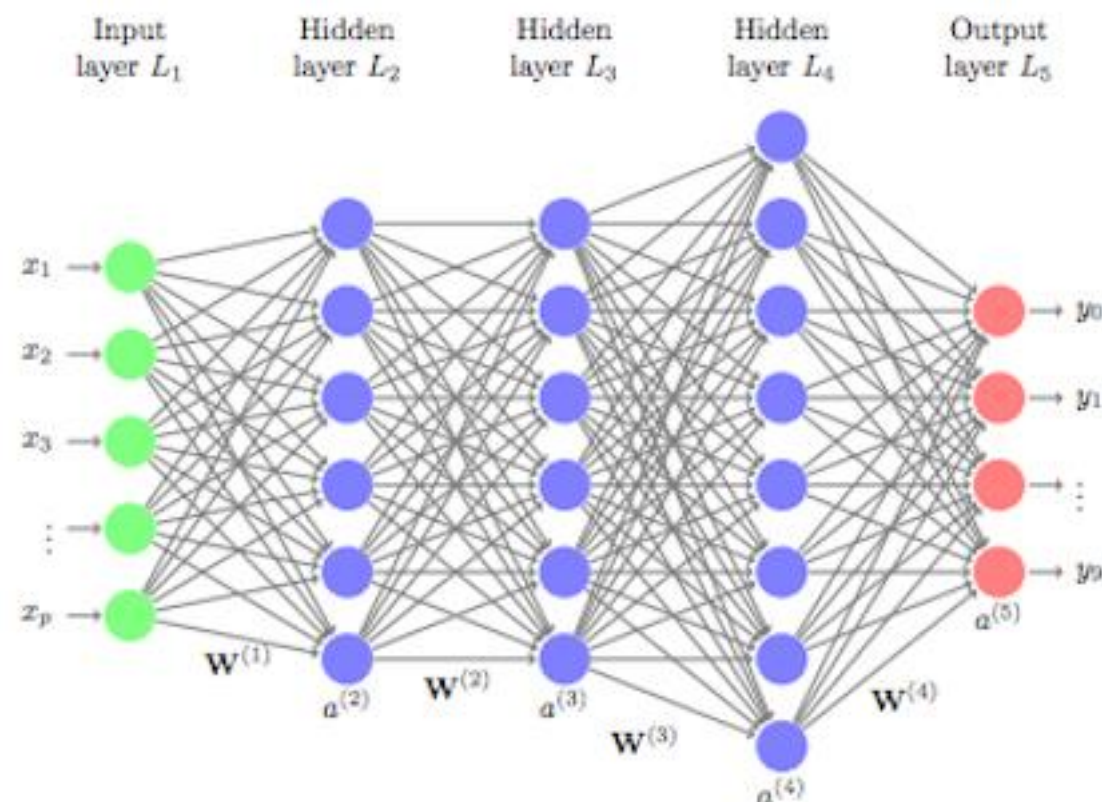


- How do we encode variable length?
 - NN have a strict fixed limit. Padding is not an option
- How do they encode position?
 - Sequence dependency is a property super important in text.
- How do we encode dependency?

How to process text in NN



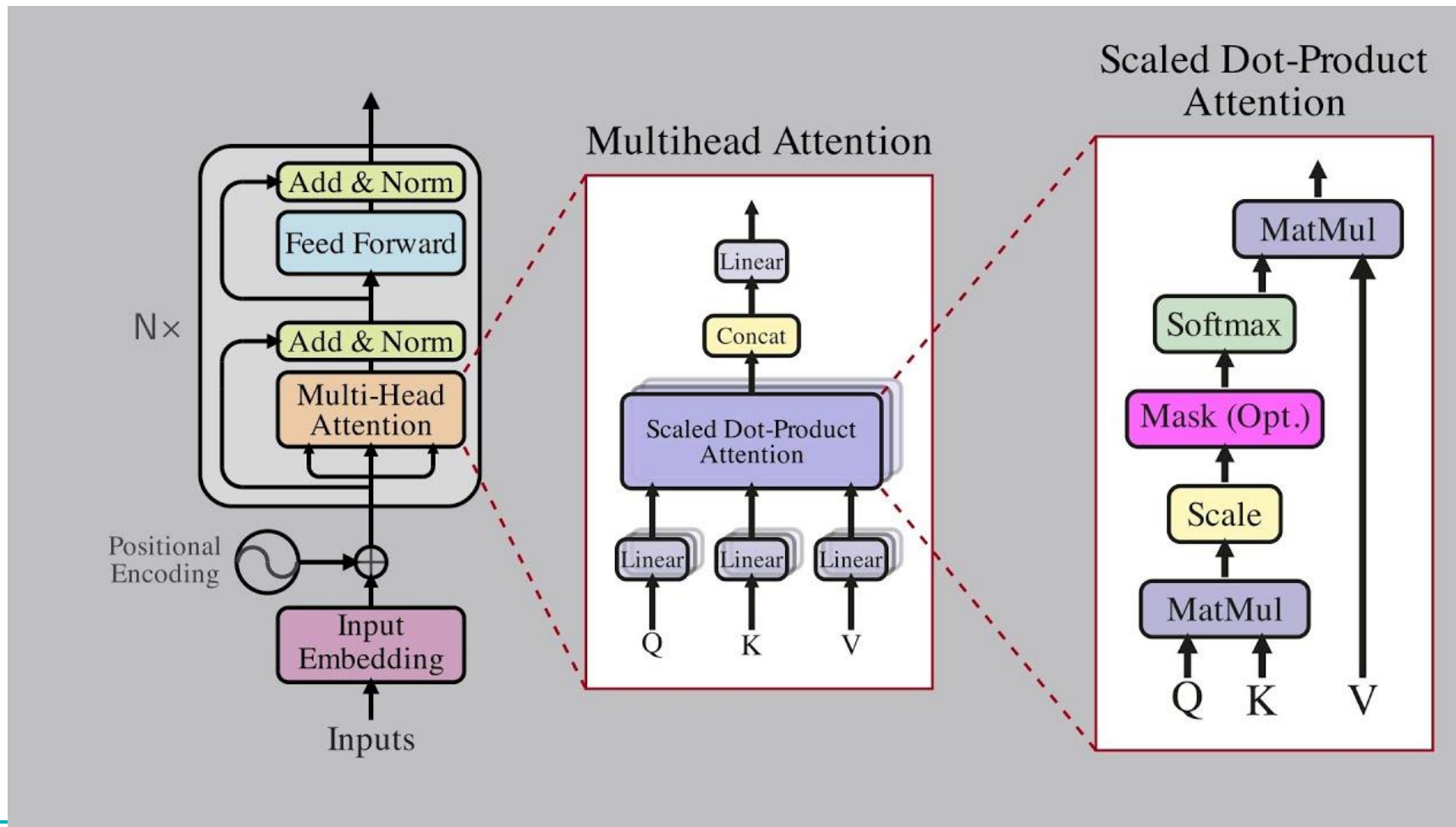
➤ Feed-forward neural network?



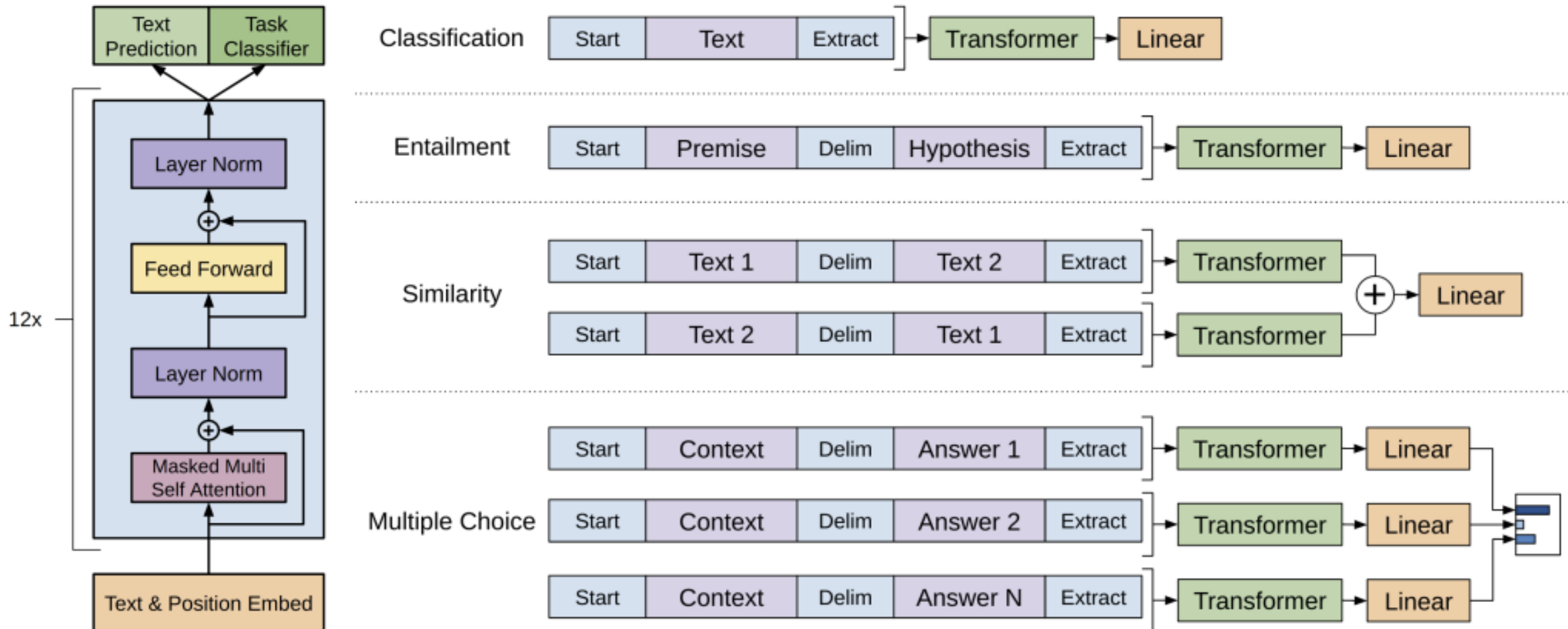
- How do we encode variable length?
 - They don't, they set maximum size not an
- How do they encode position?
 - Position encoding with embeddings super
 -
- How de we encode dependency?

Attention!

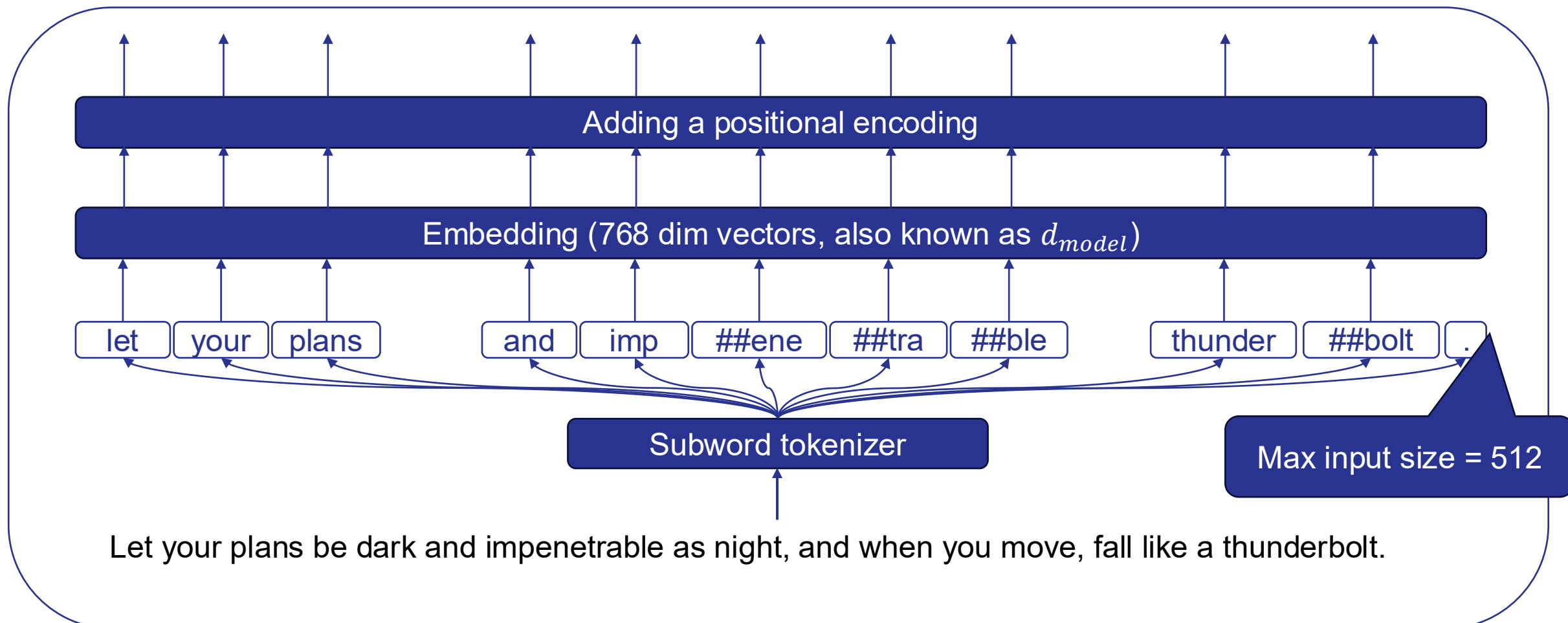
Transformer Architecture



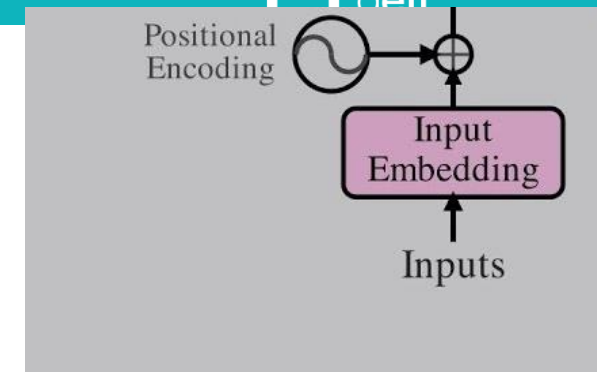
Transformer Architecture



Transformer Architecture: Input



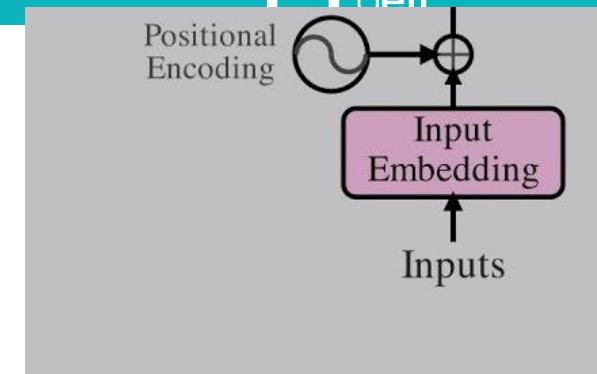
Transformer Architecture



- Positional encoding (How can we represent order?)
 - We can explicitly inject information about the position of tokens back into the inputs.
 - Intuition lets create a unique positional vector for position 1, position 2 etc... that we can add to the embeddings
- Now, the word "cat" in position 3 looks mathematically different than the word "cat" in position 5.

HOW?

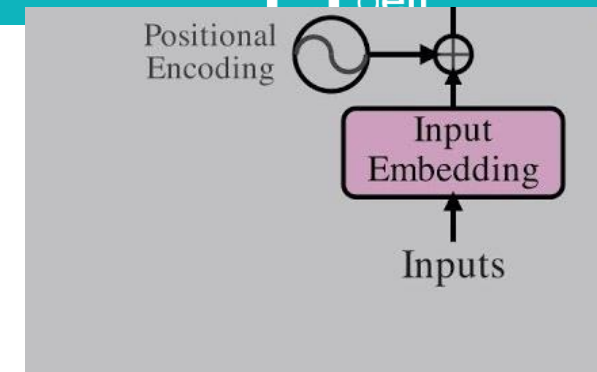
Transformer Architecture



- Consider how we count in binary
 - Note how each bit (column) changes according to a frequency.

0:	0	0	0	0	8:	1	0	0	0
1:	0	0	0	1	9:	1	0	0	1
2:	0	0	1	0	10:	1	0	1	0
3:	0	0	1	1	11:	1	0	1	1
4:	0	1	0	0	12:	1	1	0	0
5:	0	1	0	1	13:	1	1	0	1
6:	0	1	1	0	14:	1	1	1	0
7:	0	1	1	1	15:	1	1	1	1

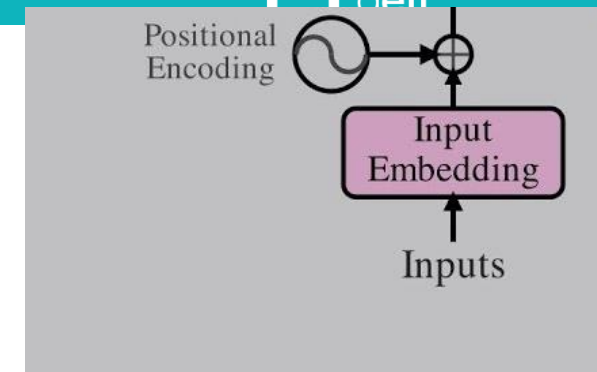
Transformer Architecture



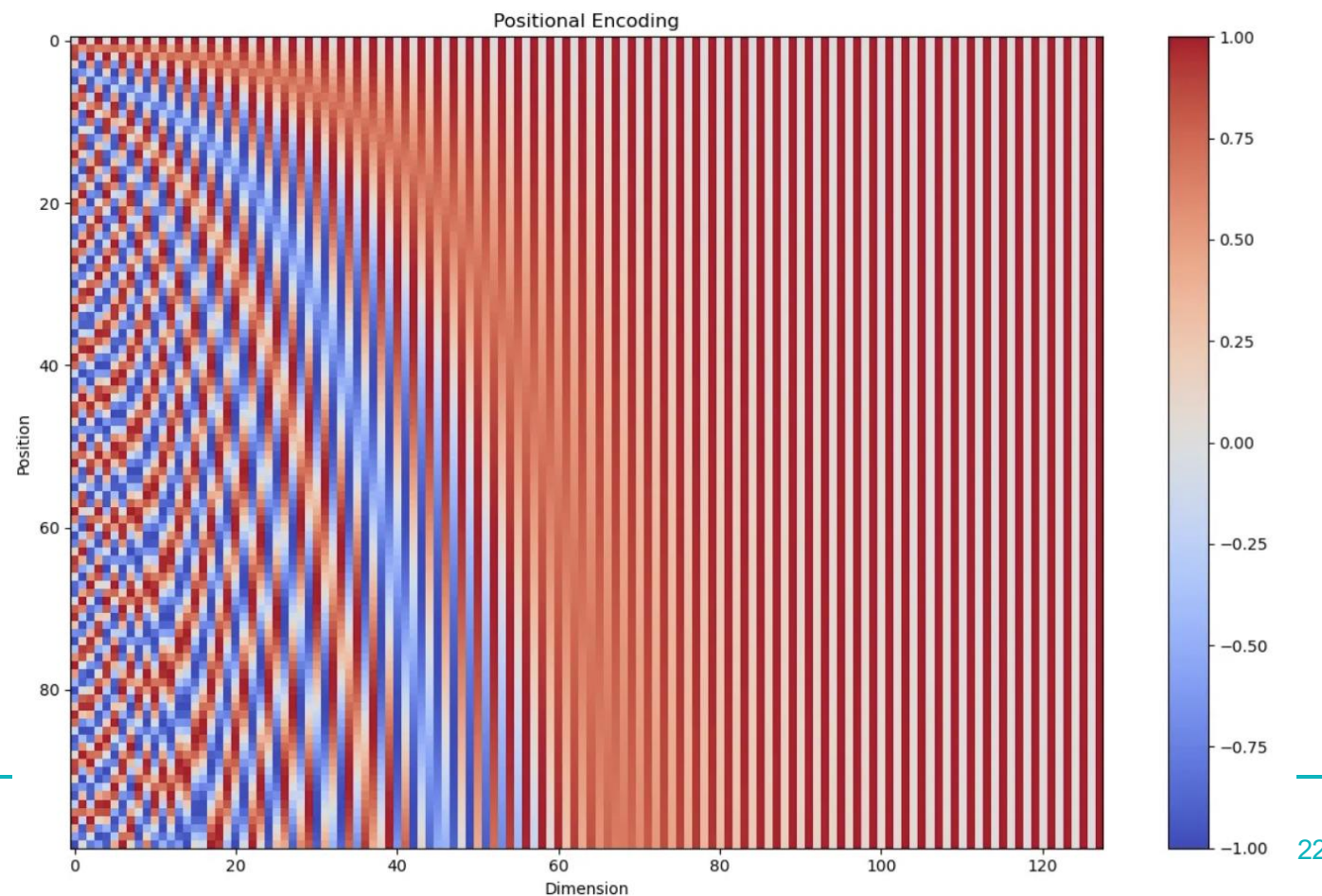
- Authors proposed a quite similar idea but for continuous space.

$$PE_{(pos,i)} = \begin{cases} \sin\left(\frac{pos}{10000^{i/d_{model}}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{pos}{10000^{(i-1)/d_{model}}}\right) & \text{if } i \text{ is odd} \end{cases}$$

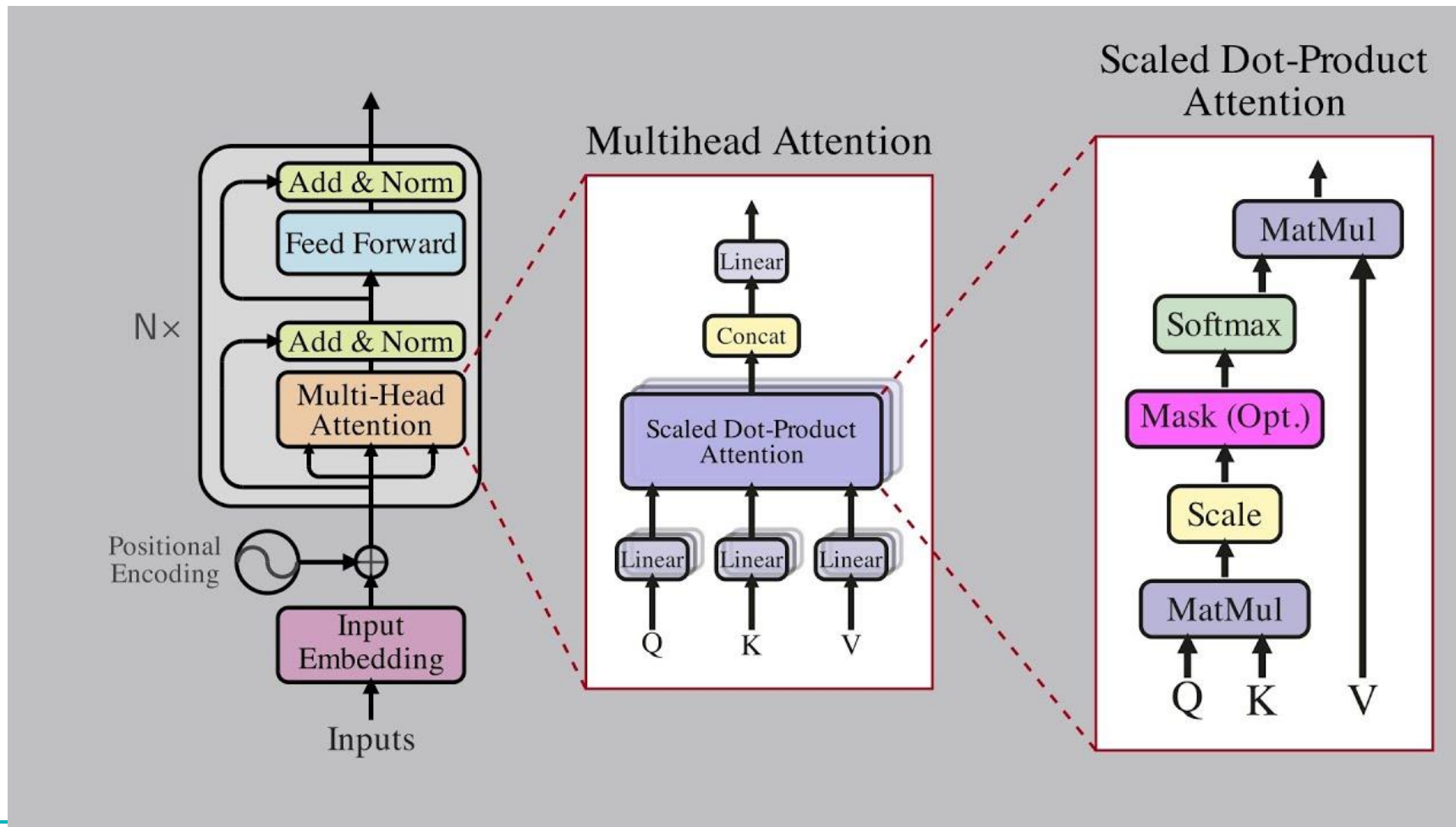
Transformer Architecture



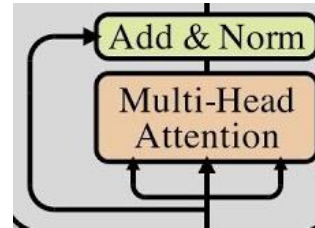
0: 0 0 0 0	8: 1 0 0 0
1: 0 0 0 1	9: 1 0 0 1
2: 0 0 1 0	10: 1 0 1 0
3: 0 0 1 1	11: 1 0 1 1
4: 0 1 0 0	12: 1 1 0 0
5: 0 1 0 1	13: 1 1 0 1
6: 0 1 1 0	14: 1 1 1 0
7: 0 1 1 1	15: 1 1 1 1



Transformer Architecture



Transformer Architecture

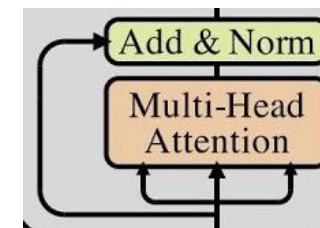


- **Multi-head attention?**
 - Like more than one? Why do we need more than one?

- **Intuition:** Words share many different types of relationships simultaneously.
 - In the sentence: *"I went to the bank to deposit money."*
 - **Syntactic:** "bank" is a noun.
 - **Semantic:** "bank" is related to finance (not a river).
 - **Positional:** "bank" comes after "the".

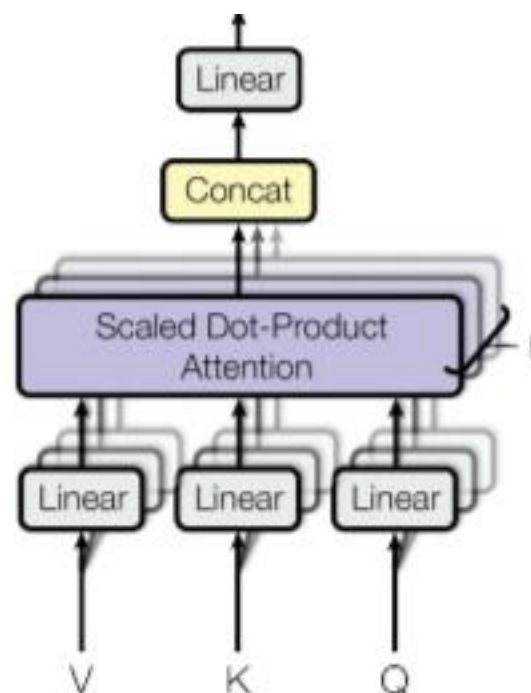
- If we only compute attention once (a single head), the model might average out these relationships and miss important nuances.
 - There is a lot of advantages in allowing the model to learn different types of attention.

Transformer Architecture

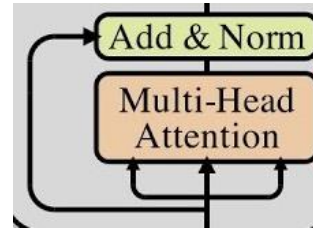


$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

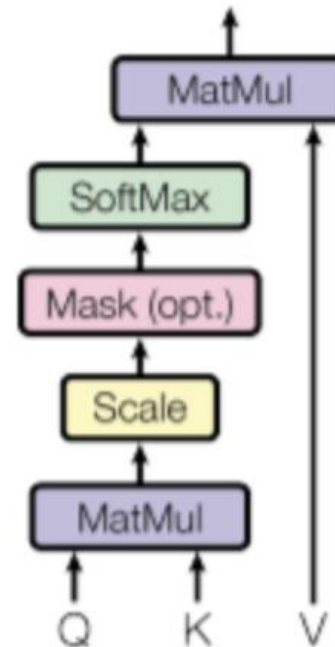
$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$



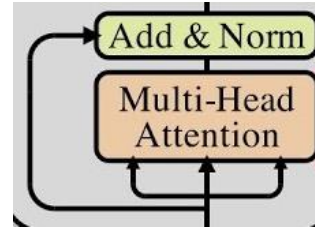
Transformer Architecture



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Transformer Architecture: Add & Norm



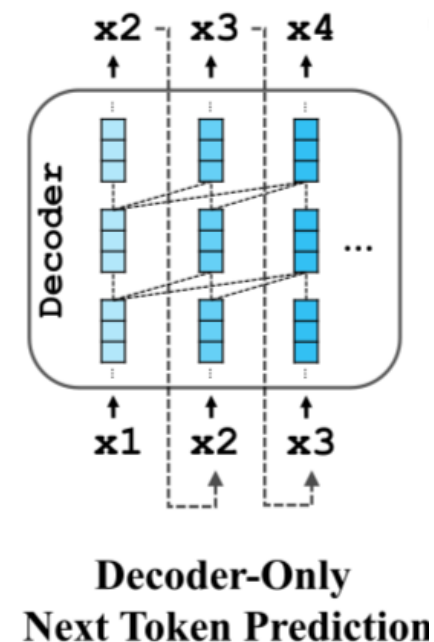
- Skip-connections (residuals): We take the original input to the layer and add it directly to the output of the Multi-Head Attention.
 - $\text{Output} = \text{Input} + \text{MHAttention}(\text{Input})$
 - **Why?** It creates a "shortcut" for the gradient during backpropagation, preventing the Vanishing Gradient problem in deep networks.
- **Layer Normalization:** Normalizes the features across the hidden dimension for each token independently.
 - Keeps values stable and significantly speeds up training.

Transformer Architecture

- After Attention figures out the relationships between words, the data passes through a **Position-wise Feed-Forward Network (FFN)**.
- It is applied to each position (token) separately and identically.
- Consists of two linear transformations with a ReLU activation in between:
 - $FFN(x) = \max(0, xW_1 + b_1) W_2 + b_2$

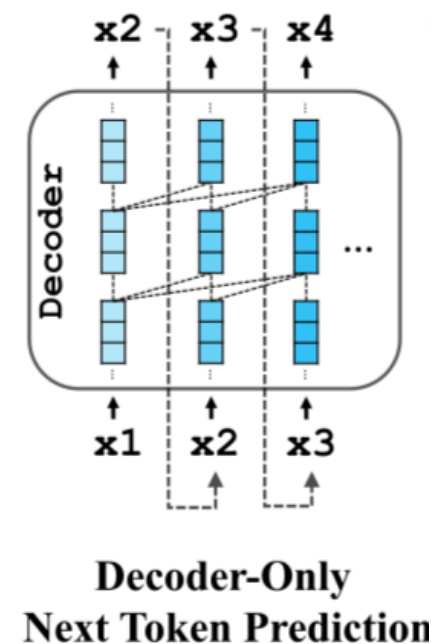
How to use LM in practice

- Transformers (specially the Decoder) generate text autoregressively
 - Feed the input prompt into the model.
 - The model outputs a probability distribution for the next word.
 - We sample a word (e.g., "apple") and append it to the input.
 - Feed the new sequence back into the model to predict the *next* word.
 - Repeat until the model outputs a special <EOS> (End of Sequence) token.



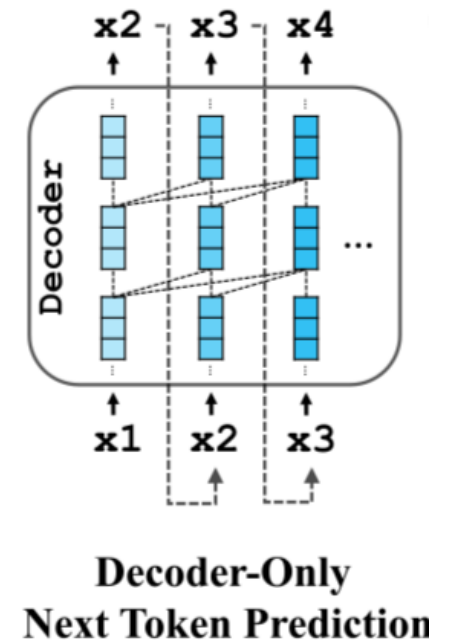
How to use LM in practice

- How can we do classification with this?
 - Training a classifier?



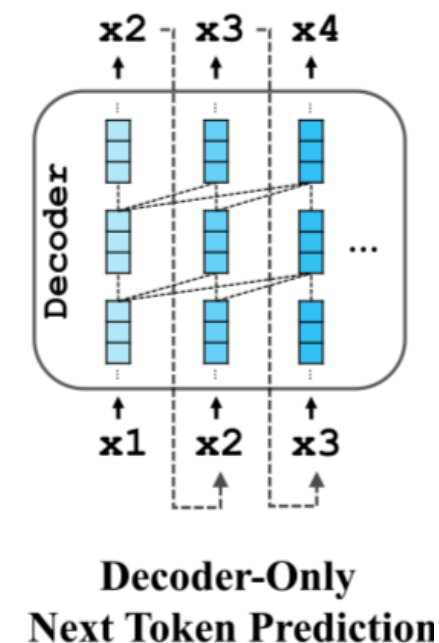
How to use LM in practice

- How can we do classification with this?
 - Training a classifier?
- Last embedding strategy
 - Extract the hidden state (embedding) of the **very last token** in the sequence.
 - Feed this single vector into a newly added, trained classification layer.
- All embeddings strategy
 - Extract the hidden states for **every single token** in the input sequence.
 - Feed those or an aggregation of those to a classification layer.



How to use LM in practice

- How can we do classification with this?
 - Training a classifier?
- Last embedding strategy
 - Extract the hidden state (embedding) of the **very last token** in the sequence.
 - Feed this single vector into a newly added, trained classification layer.
- All embeddings strategy
 - Extract the hidden states for **every single token** in the input sequence.
 - Feed those or an aggregation of those to a classification layer.



Transformer Evolution

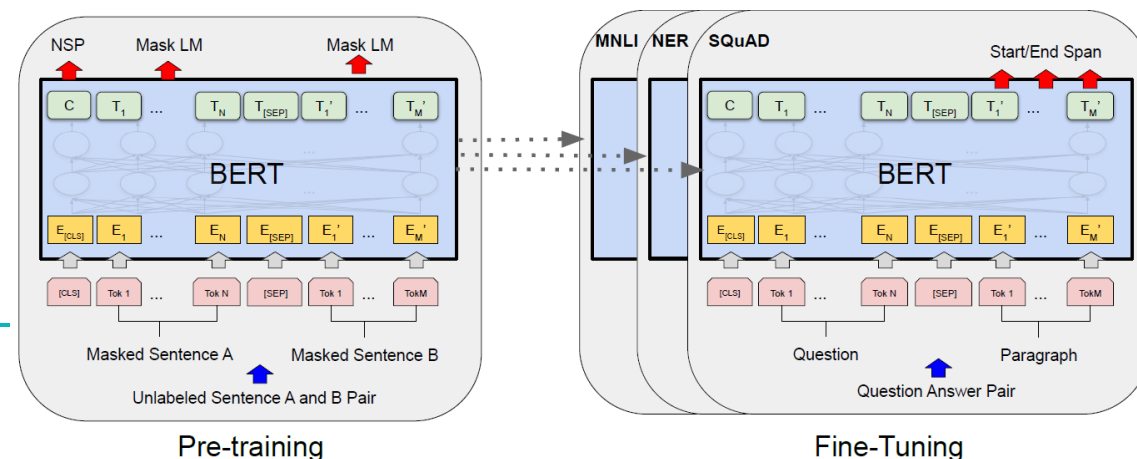
- Standard Language Models (like the Transformer Decoder) generate text autoregressively: strictly **left-to-right**.
- This is perfect for text generation. But what if our task is purely **analysis** (e.g., Sentiment Classification, Entity Recognition)?

Transformer Evolution

- Standard Language Models (like the Transformer Decoder) generate text autoregressively: strictly **left-to-right**.
- This is perfect for text generation. But what if our task is purely **analysis** (e.g., Sentiment Classification, Entity Recognition)?
- **Intuition:** Why should we be bound to left-to-right?
- **Idea:** Let's drop the Decoder entirely and just use the **Encoder**, which natively looks at the whole sequence at once. This became **BERT**.

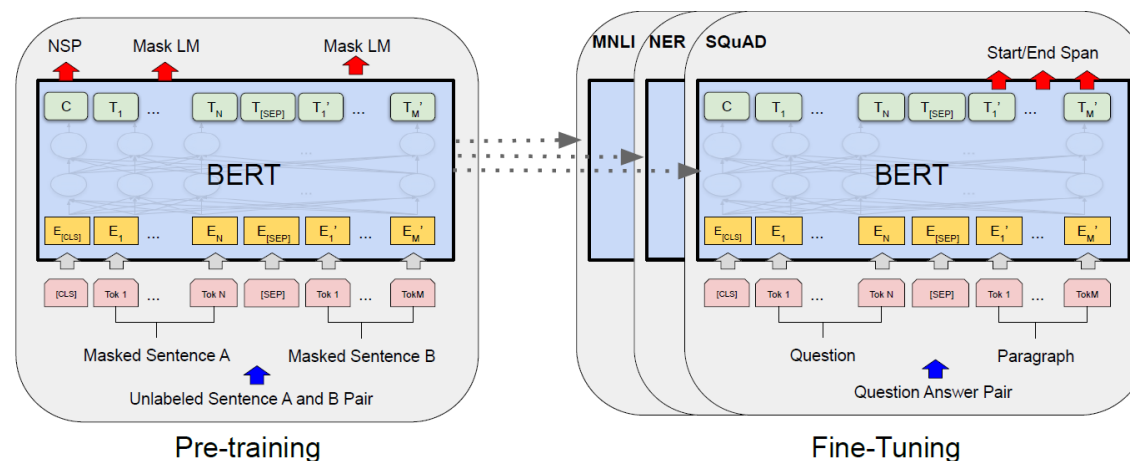
Transformer Evolution

- **BERT** (Bidirectional Encoder Representations from Transformers) is an Encoder-only model.
- **Problem:** If a model can see both left and right context, how do we train it? If we just ask it to predict the next word, it can "cheat" by looking ahead.
- **The Solution (Masked LM):** We take a sentence and randomly replace 15% of the words with a [MASK] token.
 - *Example:* "The man went to the [MASK] to buy some [MASK]."

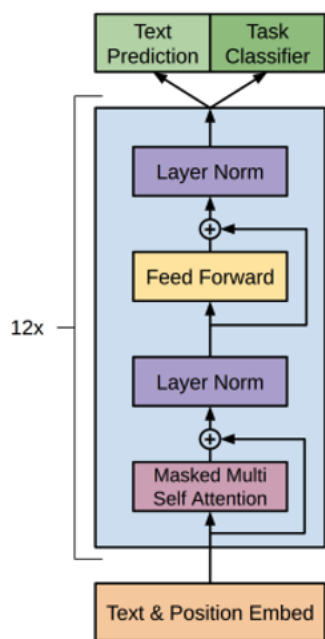


How to use BERT in practice

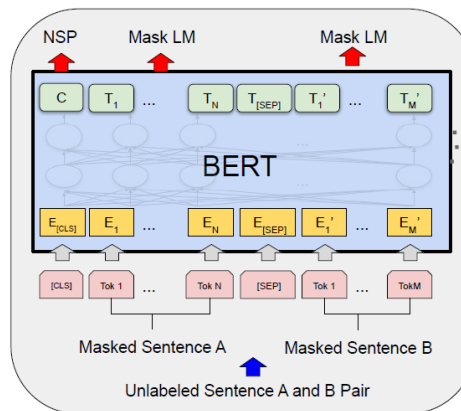
- **Step 1: Pre-training.** Train BERT on billions of words using Masked Language Modeling.
- **Step 2: Fine-Tuning.** Take this pre-trained model and adapt it to your specific task (e.g., Sentiment Analysis).



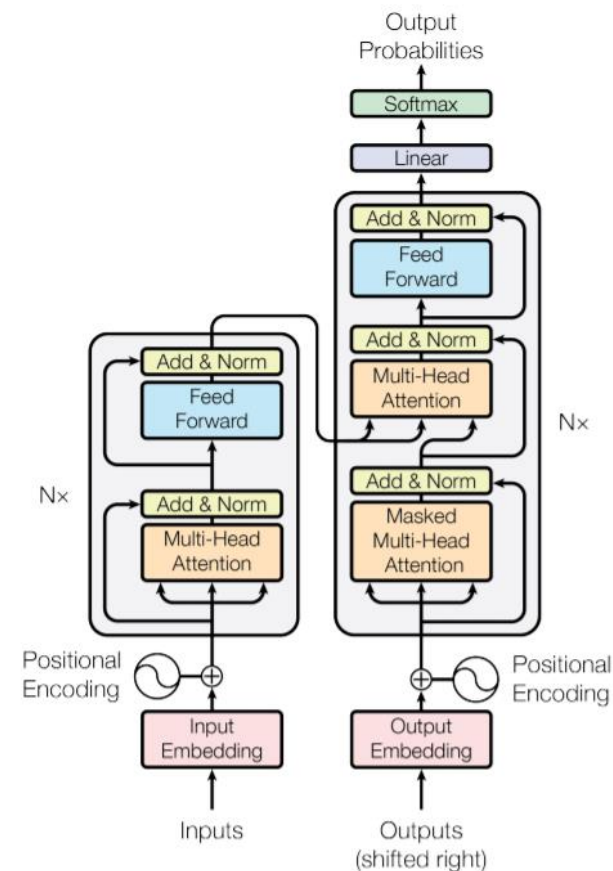
Class of transformers



Encoder Only (GPT)



Pre-training
Encoder Only (BERT)



Decoder-Encoder (T5)

Transformer Analysis and Limitation

- To compute Self-Attention, every word must look at every other word.
 - This creates an $N \times N$ attention matrix (where N is the sequence length).
- **Speed & Memory:** Complexity grows quadratically $O(N^2)$



Major step ups

- **Flash-attention:** A hardware-aware algorithm that calculates exact Attention without ever writing the massive $N \times N$ matrix to the GPU's slow memory. Massively speeds up training and allows longer contexts.
- **Sliding window attention:** Replaces global attention with a localized window. As layers stack, the model retains full document comprehension but avoids the quadratic memory explosion of standard Attention.

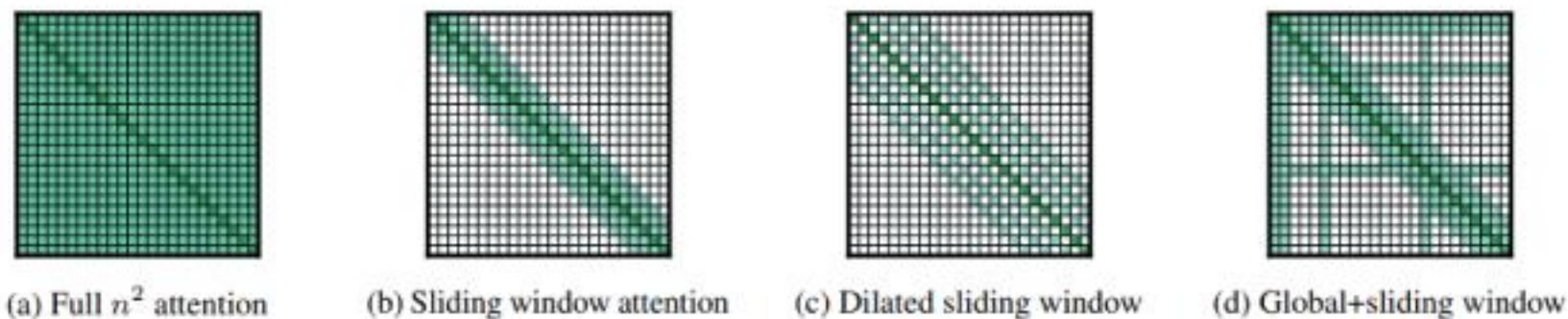


Figure 2: Comparing the full self-attention pattern and the configuration of attention patterns in our Longformer.

Transformer scaling laws

- **The Foundation of Modern LLMs:** Empirical observations showing that model performance (loss) improves predictably as we scale up.
- **The Three Pillars of Scaling:** Performance scales as a power-law relationship with:
 - **Compute:** The total processing power used during training.
 - **Model Size:** The number of parameters (neural network weights).
 - **Dataset Size:** The number of training tokens.
- The model that does everything!

We enter the era of LLMs

- A LLM is just a really big decoder-only transformer model.
 - One problem, the models are so big that inference is slow!



LLMs key improvements

- A LLM is just a really big decoder-only transformer model.
 - One problem, the models are so big that inference requires a lot of memory and is slow!
- Mixture of Experts
 - A routing architecture that only activates a fraction of the neural network for each token
- Quantization
 - Compresses the model by reducing the numerical precision of its weights (e.g., converting 16-bit floats to 8-bit or 4-bit integers).
- **KV-Caching**
 - Saves the mathematical states (Key and Value vectors) of previously processed tokens in the GPU's memory.

LLMs key improvements

➤ A LLM is just a sequence of steps

- One problem

➤ Mixture of experts

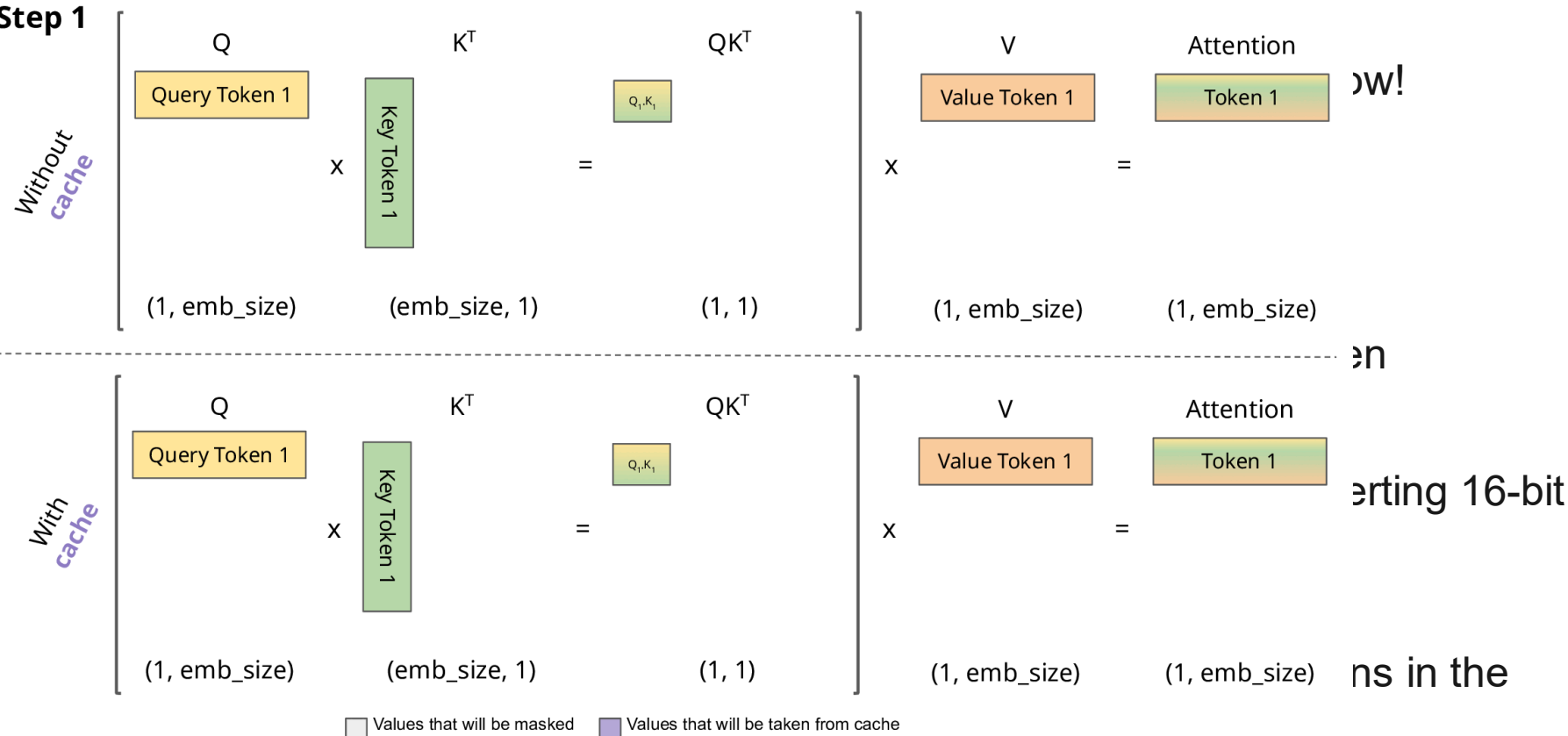
- A routing algorithm

➤ Quantization

- Compresses floats to 8-bit

➤ KV-Caching

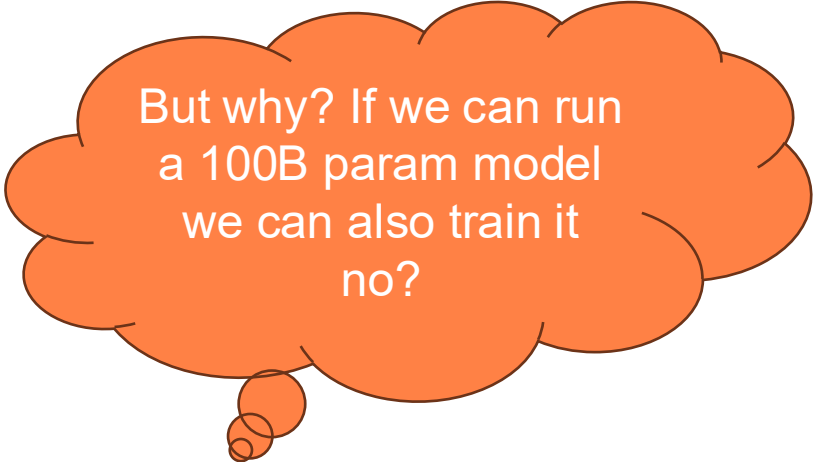
- Saves the memory on GPU's memory



3. PEFT (Adapters)

How can we finetune large models?

- Problem: modern LLMs have 7 Billion to 100+ Billion parameters.
- Doing standard fine-tuning (updating every single weight) requires massive GPU clusters and hundreds of gigabytes of VRAM.
 - It is simply impossible for normal developers or researchers to fine-tune these models on consumer hardware.



But why? If we can run
a 100B param model
we can also train it
no?

How can we finetune large models?

- No! To train a model, the GPU cannot just hold the neural network; it has to hold all the mathematical components required to update it.
- Let's look at the math
 - 2B parameter model
 - Weights: 4 bytes x 2B = 8 GB
 - Gradients: 4 bytes x 2B = 8GB
 - If we use AdamW (which is the standard)
 - Adam momentum 4 bytes x 2B = 8 GB
 - Adam variance 4 bytes x 2B = 8 GB
- So, a model that requires 8GB of VRAM to exist, it actually requires 32GB of VRAM to be trained.

How can we finetune large models?

- **The Solution:** Parameter-Efficient Fine-Tuning (PEFT). Freeze the original model and only train a tiny fraction of new parameters!
- The current industry standard is known as LoRA (Low-Rank Adaptation)

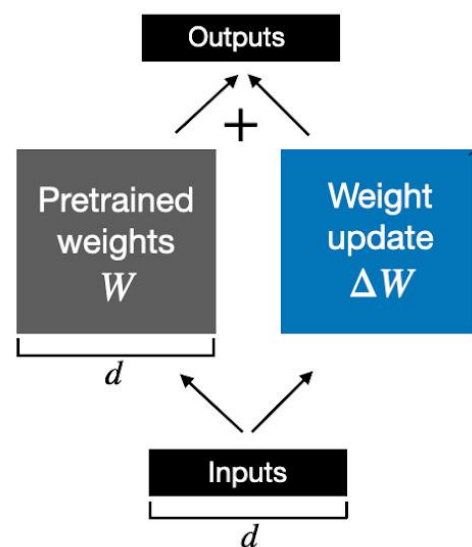
LoRA

- Observation: a weight update can be defined as $W_{new} = W_{original} + \Delta W$
- Idea: let's factorize ΔW as the product of two much smaller matrices A and B , where $A \cdot B \approx \Delta W$.

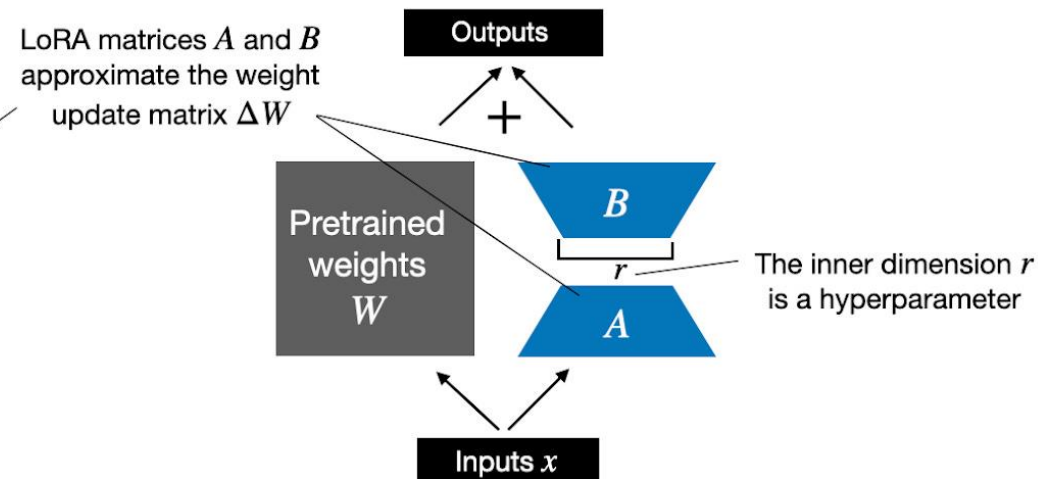
➤ Example:

- $r=8$
- If $W = 10000 \times 10000$
- $A = 10000 \times 8$
- $B = 8 \times 10000$
- We went from 100M to 160k

Weight update in **regular finetuning**



Weight update in **LoRA**



LoRA

- Going back to the same example, now to train the same model with LoRA we need:
 - 2B parameter model
 - Weights: 4 bytes x 2B = 8 GB
 - Lora Weights: 4 bytes x 3.2M = 12.8MB (3.2M (2B*0.0016))
 - Lora Gradients: 4 bytes x 3.2M = 12.8MB
 - If we use AdamW (which is the standard)
 - Adam momentum 4 bytes x 3.2M = 12.8MB
 - Adam variance 4 bytes x 3.2M = 12.8MB
- Now we only need 8.05GB approximately.

LoRA in practice

- The most streamline way of setting up LoRA on HuggingFace.

```
model = AutoModelForCausalLM.from_pretrained(model_id, device_map="auto")

# 2. Define the LoRA configuration
# This targets specific layers and reduces the number of trainable parameters
peft_config = LoraConfig(
    r=16,                # higher means more trainable param - typically between 4-32
    lora_alpha=32,        # Scaling factor - typically 2x rank
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)

# 4. Set up the Trainer
training_args = TrainingArguments(
    output_dir="./lora-test",
    per_device_train_batch_size=4,
    num_train_epochs=1,
    learning_rate=2e-4, # LoRA usually needs a slightly higher learning rate
    logging_steps=10,
    push_to_hub=False,
)

trainer = SFTTrainer(
    model=model,
    train_dataset=dataset,
    dataset_text_field="text",
    peft_config=peft_config,
    args=training_args,
)

# 5. Train!
trainer.train()
```



15 minutes

Hands-On Lab

Thank You!

See you next week!

Lab submission due in 1 week
